K8S-05: Workload Monitoring

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 5 of 14 | **Created:** January 2026 | **Last Updated:** 10/15/2026

Application-Level Observability in Kubernetes

Workload monitoring focuses on the application layer: deployments, pods, containers, and the services they provide. This notebook covers monitoring Kubernetes workloads from deployment health to service performance.

Table of Contents

1. Workload Types and Monitoring
2. Deployment Health
3. Pod and Container Metrics
4. Service Performance
5. Distributed Tracing in K8s
6. Application Logs
7. Workload Anomalies

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring
DynaKube	cloudNativeFullStack or applicationMonitoring

Requirement	Details
Permissions	`storage:metrics:read`, `storage:spans:read`, `storage:logs:read`, `storage:events:read`, `storage:smartscape:read`
Knowledge	K8S-01 Fundamentals, K8S-04 Cluster Monitoring

1. Workload Types and Monitoring

Kubernetes Workload Controllers

Workload Type	Use Case	Monitoring Focus
Deployment	Stateless apps	Replica availability, rollout status
StatefulSet	Stateful apps (DBs)	Pod identity, persistent storage
DaemonSet	Node-level agents	Coverage across all nodes
Job	Batch processing	Completion rate, duration
CronJob	Scheduled tasks	Execution timing, failures

Dynatrace Workload Entities

Smartscape node	Classic entity	Kubernetes resource
`K8S_DEPLOYMENT`, `K8S_STATEFULSET`, `K8S_DAEMONSET`, `K8S_REPLICASET`, `K8S_JOB`, `K8S_CRONJOB`	`CLOUD_APPLICATION` (one classic type for every workload kind)	The workload controllers
`K8S_NAMESPACE`	`CLOUD_APPLICATION_NAMESPACE`	Namespace
`K8S_POD`	`CLOUD_APPLICATION_INSTANCE`	Pod
`CONTAINER`	`CONTAINER_GROUP_INSTANCE`	Container

Smartscape node	Classic entity	Kubernetes resource
`SERVICE`	`SERVICE`	A Dynatrace-detected service — not the Kubernetes `Service` object (`K8S_SERVICE`)

In metrics, spans and logs the workload is identified by the dimensions `k8s.workload.name` and `k8s.workload.kind`, which cover every workload kind.

```
// List Kubernetes workloads (Deployments) via smartscape topology
smartscapeNodes "K8S_DEPLOYMENT"
| fields entity.name = name, tags
| sort entity.name asc
| limit 50

// For StatefulSets, DaemonSets, etc., substitute the type:
// smartscapeNodes "K8S_STATEFULSET"
// smartscapeNodes "K8S_DAEMONSET"

// Legacy alternative (deprecated for new content):
// fetch dt.entity.cloud_application
// | fields entity.name, tags
// | sort entity.name asc
// | limit 50
```

2. Deployment Health

Deployment Status Indicators

Metric	Description	Healthy State
Available Replicas	Pods passing readiness	Equals desired
Ready Replicas	Pods passing all probes	Equals desired
Updated Replicas	Pods with latest spec	Equals desired
Collision Count	Hash collisions	Zero

Dynatrace records the Deployment conditions as the metric `dt.kubernetes.workload.conditions` (dimensions `workload_condition`, boolean `condition_status`). A workload whose `Available` condition is false has fewer available replicas than its minimum — the query below lists them.

Rollout Monitoring

During deployments, monitor:
- Old ReplicaSet scaling down
- New ReplicaSet scaling up
- Pod readiness during transition

```
// Workloads whose Available condition is false in the last hour
// dt.kubernetes.workload.conditions writes 1 per (workload, condition,
status) present.
// bucketsUnavailable counts time buckets (one minute each at from:-1h).
Executed 10/02/2026.
timeseries c = max(dt.kubernetes.workload.conditions), from:-1h,
  by:{k8s.cluster.name, k8s.namespace.name, k8s.workload.kind,
k8s.workload.name, workload_condition, condition_status}
| filter workload_condition == "Available" and condition_status == false
| fieldsAdd bucketsUnavailable = arraySize(arrayRemoveNulls(c))
| fields k8s.cluster.name, k8s.namespace.name, k8s.workload.kind,
k8s.workload.name, bucketsUnavailable
| sort bucketsUnavailable desc
```

```
// Workload lifecycle events - track rollouts and restarts
// Data object corrected 08/12/2026. Kubernetes events are NOT logs. This cell
scraped
// `fetch logs` for `log.source` containing "kubernetes" or for content
substrings like "BackOff" —
// no log.source matches, and kubelet event text is not in the log stream, so
it returned nothing
// while the cluster emitted 231,296 Kubernetes event records in the same
window.
// They arrive as `fetch events | filter event.provider ==
"KUBERNETES_EVENT"`, STRUCTURED:
//   dt.kubernetes.event.reason            Unhealthy · BackOff · Killing ·
FailedScheduling ·
//                                         FailedMount · BackoffLimitExceeded
· EvictionThresholdMet …
//   dt.kubernetes.event.message           the human-readable text
//   status                                "WARN" for Kubernetes Warning
events, "INFO" for Normal
//   dt.kubernetes.event.involved_object.kind / .name
//   dt.kubernetes.event.count / .first_seen / .last_seen
//   plus k8s.cluster.name · k8s.namespace.name · k8s.pod.name ·
k8s.workload.name · k8s.node.name
// NOTE: event.type is CUSTOM_INFO on every one of these — it is NOT
"Warning". The Warning/Normal
// split is in status ("WARN" / "INFO"). dt.kubernetes.event.important was
"true" on all 204,078
// event records over 30 days (10/02/2026), so it separates nothing. Enumerate
reasons with:
//   fetch events, from:-24h | filter event.provider == "KUBERNETES_EVENT"
//   | summarize n = countDistinctExact(event.id), by:
{dt.kubernetes.event.reason} | sort n desc
// Each open event is re-written about every 5 min under the same event.id
with a cumulative
// dt.kubernetes.event.count — count events with countDistinctExact(event.id),
never count().
fetch events, from:-6h
| filter event.provider == "KUBERNETES_EVENT"
| filter isNotNull(k8s.workload.name)
| summarize events = countDistinctExact(event.id), by:{k8s.namespace.name,
k8s.workload.name, dt.kubernetes.event.reason}
| sort events desc
| limit 25
```

```
// Container restarts per pod - identify unstable workloads
// Use the restart counter, not events: a BackOff or Unhealthy event is not a
restart, and one
// restart loop can produce many events (or few, since Kubernetes aggregates
repeats into one
// event with a rising count). Executed 10/02/2026.
timeseries r = sum(dt.kubernetes.container.restarts), from:-24h,
  by:{k8s.cluster.name, k8s.namespace.name, k8s.workload.name, k8s.pod.name}
| fieldsAdd restarts = arraySum(r)
| filter restarts &gt; 0
| fields k8s.cluster.name, k8s.namespace.name, k8s.workload.name,
k8s.pod.name, restarts
| sort restarts desc
| limit 25
```

3. Pod and Container Metrics

Container Resource Metrics

Metric	Description	Alert Threshold
CPU Usage %	Usage vs. limit	>85% sustained
Memory Usage %	Usage vs. limit	>90%
CPU Throttled	Time throttled	>10% of time
Memory Working Set	Active memory	Approaching limit

Pod Lifecycle States

State	Description	Action
Pending	Waiting to schedule	Check resources, node selectors
Running	At least one container running	Normal
Succeeded	All containers exited 0	Job completed
Failed	Containers exited non-zero	Check logs
Unknown	Node communication lost	Check node health

```
// Container CPU usage — high consumers
// Sort on a scalar. `sort` on the timeseries array itself compares arrays element by element,
// so the order has nothing to do with the average — reduce with arrayAvg() first.
timeseries cpu = avg(dt.kubernetes.container.cpu_usage), from:-1h,
  by:{k8s.cluster.name, k8s.namespace.name, k8s.pod.name, k8s.container.name}
| fieldsAdd avgCpuMillicores = round(arrayAvg(cpu), decimals: 0)
| fields k8s.cluster.name, k8s.namespace.name, k8s.pod.name, k8s.container.name, avgCpuMillicores
| sort avgCpuMillicores desc
| limit 15
```

```
// Container memory usage approaching limits
// "Approaching the limit" needs the limit: compare working set with limits_memory per container.
// Containers with no memory limit drop out (limitMiB is null) — they cannot be OOM-killed for
// exceeding a limit, only by node pressure.
timeseries {
    used = avg(dt.kubernetes.container.memory_working_set),
    lim = avg(dt.kubernetes.container.limits_memory)
  }, from:-1h, by:{k8s.cluster.name, k8s.namespace.name, k8s.pod.name, k8s.container.name}
| fieldsAdd usedMiB = round(arrayAvg(used) / 1048576, decimals: 0),
            limitMiB = round(arrayAvg(lim) / 1048576, decimals: 0)
| filter limitMiB &gt; 0
| fieldsAdd pctOfLimit = round(100 * usedMiB / limitMiB, decimals: 1)
| fields k8s.cluster.name, k8s.namespace.name, k8s.pod.name, k8s.container.name, usedMiB, limitMiB, pctOfLimit
| sort pctOfLimit desc
| limit 15
```

```
// Container CPU throttling - performance impact
timeseries avgThrottled = avg(dt.containers.cpu.throttled_time), from:-1h, by:{dt.entity.container_group_instance}
| fieldsAdd avgThrottledValue = arrayAvg(avgThrottled)
| filter avgThrottledValue &gt; 0
| sort avgThrottledValue desc
| limit 15
```

4. Service Performance

Service-Level Metrics

Metric	Description	SLO Example
Response Time	P50, P90, P99 latency	P99 < 500ms
Error Rate	Failed requests %	< 0.1%
Throughput	Requests per second	Based on capacity
Availability	Successful responses	> 99.9%

Golden Signals for Services

Signal	What to Monitor	DQL Approach
Latency	Response time distribution	Timeseries with percentiles
Traffic	Request rate	Count over time
Errors	Error rate, types	`countIf(span.status_code == "error")`
Saturation	Resource utilization	CPU, memory, connections

> **Span failure status: three traps, all silent.** Getting error rate right on spans depends on three facts, and each of them fails by returning a plausible number rather than an error.
>
> | Trap | What happens |
> |---|---|
> | `otel.status_code` | **The field does not exist** — it has no row in `dt.semantic_dictionary.fields`. The real field is **`span.status_code`** (`stable`). Likewise `otel.status_message` → `span.status_message` (`experimental`, and frequently null even on failing spans). |

Trap	What happens
`== "ERROR"`	Values are **lowercase**. On a live tenant `span.status_code == "ERROR"` matched **0** spans against **18,630** for `"error"` in the same two-hour window (08/11/2026). A rename without the case fix converts a hard failure into a silent zero.
`countIf(span.status_code != "error")`	**`span.status_code` is null on virtually every successful span** — `"ok"` is written so rarely it is noise (9 spans out of ~659,000 in that window). A `!=` comparison against null yields null, not true, so this counts almost nothing. Derive successes as **`total - errors`**.

The third trap is the expensive one: an availability SLO built on `countIf(… != "error")` reads **0% available** on a healthy service, and nothing in the query errors to tell you.

```
// Service response time (server spans), slowest p99 first
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    p50_ms = percentile(duration, 50) / 1ms,
    p90_ms = percentile(duration, 90) / 1ms,
    p99_ms = percentile(duration, 99) / 1ms
  }, by:{dt.service.name}
| sort p99_ms desc
| limit 15
```

```
// Service error rates
// The field is span.status_code (stable). otel.status_code does not exist.
// Values are lowercase ("error"), and the field is NULL on successful spans —
// so successes must be derived as total - errors, never counted directly.
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {
    total = count(),
    errors = countIf(span.status_code == "error")
  }, by:{dt.entity.service}
| fieldsAdd successes = total - errors
| fieldsAdd errorRate = round(100.0 * errors / total, decimals: 2)
| filter errors &gt; 0
| sort errorRate desc
| limit 15
```

```
// Service throughput
fetch spans, from:-1h
| filter span.kind == "server"
| summarize requestCount = count(), by:{dt.entity.service}
| sort requestCount desc
| limit 10
```

5. Distributed Tracing in K8s

Trace Context in Kubernetes

Distributed tracing in Kubernetes follows requests across:
- Ingress → Services
- Service → Service (east-west traffic)
- Service → External (databases, APIs)

Trace Attributes in K8s

Attribute	Source	Use Case
`k8s.cluster.name`	OneAgent	Filter by cluster
`k8s.namespace.name`	OneAgent	Filter by namespace

Attribute	Source	Use Case
`k8s.workload.name`	OneAgent	Identify workload (any kind — use `k8s.workload.kind` to tell them apart)
`k8s.pod.name`	OneAgent	Instance-level analysis
`k8s.container.name`	OneAgent	Container identification

On the validation tenant (10/02/2026) every span carrying `k8s.namespace.name` also carried the four fields above, and none carried `k8s.deployment.name` — group spans by `k8s.workload.name`.

```
// Spans by namespace — span volume, not request count (one request produces many spans)
fetch spans, from:-1h
| filter isNotNull(k8s.namespace.name)
| summarize {spanCount = count(), avgDurationMs = avg(duration) / 1ms}, by: {k8s.namespace.name}
| sort spanCount desc
| limit 15
```

```
// Slow traces by workload
fetch spans, from:-1h
| filter span.kind == "server" and duration &gt; 1s
| filter isNotNull(k8s.workload.name)
| fields start_time, trace.id, k8s.namespace.name, k8s.workload.name, duration
| sort duration desc
| limit 20
```

6. Application Logs

Container Log Collection

OneAgent collects container logs from:
- stdout/stderr streams
- Mounted log files (with configuration)

Log Attributes

Attribute	Description
`k8s.namespace.name`	Source namespace
`k8s.pod.name`	Source pod
`k8s.container.name`	Source container
`dt.entity.container_group_instance`	Entity relationship

```
// Application error logs by namespace
fetch logs, from:-1h
| filter loglevel == "ERROR" or loglevel == "SEVERE"
| filter isNotNull(k8s.namespace.name)
| summarize errorCount = count(), by:{k8s.namespace.name}
| sort errorCount desc
| limit 15
```

```
// Recent application logs with context
fetch logs, from:-1h
| filter isNotNull(k8s.namespace.name)
// status == "ERROR" also counts SEVERE, CRITICAL and FATAL logs; loglevel ==
"ERROR" misses them
| filter status == "ERROR"
| fields timestamp, k8s.namespace.name, k8s.pod.name, content
| sort timestamp desc
| limit 30
```

7. Workload Anomalies

Common Anomaly Types

Anomaly	Indicators	Root Cause
Memory Leak	Steadily increasing memory	Application bug
CPU Spike	Sudden CPU increase	Traffic surge, infinite loop
Error Storm	Rapid error increase	Dependency failure

Anomaly	Indicators	Root Cause
Latency Degradation	Increasing response time	Resource contention

Dynatrace Intelligence for Workloads

Dynatrace Intelligence automatically detects:
- Response time degradation
- Error rate increases
- Resource saturation
- Failure rate anomalies

```
// Error count by namespace (detecting increases)
fetch logs, from:-1h
// status == "ERROR" also counts SEVERE, CRITICAL and FATAL logs; loglevel ==
"ERROR" misses them
| filter status == "ERROR"
| filter isNotNull(k8s.namespace.name)
| summarize errorCount = count(), by:{k8s.namespace.name}
| sort errorCount desc
| limit 10
```

```
// Memory usage by workload (detecting high consumers)
// Workload memory is the sum of its containers (and replicas). rollup: avg
averages each container
// within a time bucket before the cross-container sum. Sort on the scalar,
never on the array.
timeseries mem = sum(dt.kubernetes.container.memory_working_set, rollup: avg),
from:-1h,
  by:{k8s.cluster.name, k8s.namespace.name, k8s.workload.name}
| fieldsAdd avgMemMiB = round(arrayAvg(mem) / 1048576, decimals: 0)
| fields k8s.cluster.name, k8s.namespace.name, k8s.workload.name, avgMemMiB
| sort avgMemMiB desc
| limit 10
```

Next Steps

With workload monitoring configured, proceed to:

Next Notebook	Topic
K8S-06: Namespace Organization	Boundaries and access control
K8S-07: Events and Logs	Kubernetes event analysis
K8S-08: DQL for Kubernetes	Advanced query patterns

Summary

In this notebook, you learned:

- Kubernetes workload types and their monitoring focus
- Deployment health indicators and rollout monitoring
- Pod and container resource metrics
- Service-level performance monitoring (golden signals)
- Distributed tracing with Kubernetes attributes
- Application log analysis by namespace and pod
- Workload anomaly detection patterns

References

- Set up Dynatrace on Kubernetes (DT docs)
- Full observability deployment (DT docs)
- Application observability deployment (DT docs)
- Kubernetes app — workloads view (DT docs)
- Container platform monitoring (DT docs)
- Service monitoring (DT docs)
- smartscapeNodes command (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.